BrandGuard AI

Intelligent Brand-Governance Platform

Round 2 — The Build | Submission Document

Team Innovex | Team Leader: Ajay Kale | November 2026

1. Submission Links

Resource	URL / Details
🌐 Live Project (Frontend)	https://brandguard-frontend-6swq.onrender.com
⚙️ Live API (Backend)	https://brandguard-api-zfcd.onrender.com
📖 API Documentation	https://brandguard-api-zfcd.onrender.com/docs
💻 Source Code Repository	https://github.com/AJJU3836/BrandGuard-AI
📁 Google Drive	https://drive.google.com/drive/folders/1ybSTBQjKro8z8dBaycZW9Flvnrpbdp3r

Demo Login Credentials:

Role	Email	Password
Admin	kaleajay626@gmail.com	password123
Manager	mahesh@innovex.com	password123
Reviewer	asma@innovex.com	password123
Creator	shrushti@innovex.com	password123

2. List of Completed Features

2.1 Core Brand Review Engine

- **Banned-term detection:** Flags exact and case-insensitive matches from the brand kit's banned terminology list with error severity and suggested replacement.
- **Unsupported-claim detection:** Identifies superlative claims (#1, guaranteed, 100% safe, miracle, magic) not found in the approved facts list; maps to ASCI guidelines.
- **Fact verification:** Cross-checks content against the approved facts list; flags statements that contradict or are absent from verified brand facts.
- **Tone analysis:** Rule-based detection of tone mismatches (e.g. hyperbolic or unprofessional language) against configured tone rules.
- **Semantic guideline search:** Vector-indexed brand guidelines (TF-IDF fallback) for semantic retrieval of relevant clauses to support AI analysis.
- **Image review:** Accepts uploaded images; checks colour palette compliance and flags non-brand colours with hex values.
- **Finding severity:** All findings carry a severity (error / warning / info), category, highlighted text, guideline citation, and suggested correction.
- **AI vs. rule badge:** Every finding is labelled with its source — Rule Engine or AI — so reviewers know the confidence level immediately.
- **Export findings:** Findings exportable as JSON or CSV per draft via the review export endpoint.

2.2 Brand Kit Management

- Upload and store brand guidelines (raw text), approved terminology, banned terminology, tone rules, approved facts and colour palette.
- Brand kit indexing: guidelines chunked and indexed for semantic retrieval (TF-IDF cosine similarity).
- Per-workspace isolation: each workspace has its own brand kit; no data shared across workspaces.

- Brand kit update and re-indexing via API.

2.3 Draft Workflow & Collaboration

- Create, read, update drafts with title, content, and content-type (social post, email, blog, press release).
- Full status workflow: Draft → In Review → Approved / Rejected.
- Version history: every content save creates an immutable version snapshot with timestamp and author.
- Reviewer comments: threaded comments per draft, visible to all workspace members.
- Finding status management: each finding can be marked open, accepted, or dismissed by a reviewer.
- Approval flow: Manager/Admin can approve or reject after review.

2.4 Role-Based Access Control

Role	Capabilities
Admin	Full access: manage workspace, brand kit, all drafts, all users, audit log
Manager	Approve/reject drafts, manage brand kit, view all drafts and insights
Reviewer	Review drafts, add comments, accept/dismiss findings, cannot approve
Creator	Create and submit drafts, view own findings, cannot modify brand kit

2.5 Dashboard & Insights

- Real-time stats: total drafts, breakdown by status (draft / in_review / approved / rejected).
- Findings by severity: error, warning, info counts at a glance.
- Findings by category: terminology, claim, tone, visual breakdown.
- Top recurring issues: most frequent violation explanations across all drafts.
- Audit log: immutable record of every action (draft created, reviewed, approved, finding dismissed) with user, timestamp and AI service used.

2.6 Frontend & UX

- React + Vite SPA with Tailwind CSS — responsive layout for desktop and tablet.
- Findings panel with colour-coded severity badges (red = error, amber = warning, blue = info).
- AI vs. Rule Engine source badges with tooltip explanation on every finding.
- Image upload UI with visual findings overlay.
- Findings filter by severity and category; export to CSV/JSON from the UI.
- Dark/light mode toggle.
- Fully keyboard-navigable reviewer interface.

2.7 Infrastructure & Deployment

- Backend deployed on Render (Singapore region) — always-on free tier.
- Frontend deployed as a Render Static Site — global CDN with SPA routing.
- PostgreSQL database on Render (free tier, Singapore) — persistent across deploys.

- Auto-deploy on GitHub push to main branch via Render + GitHub integration.
- Environment variables managed via Render dashboard (DATABASE_URL, SECRET_KEY, GEMINI_API_KEY, FRONTEND_URL).
- CORS configured for production frontend URL and localhost for development.

3. Technology & Implementation Details

3.1 Technology Stack

Layer	Technology / Library	Purpose
Frontend	React 18 + Vite	SPA framework, fast dev build
Frontend	Tailwind CSS	Utility-first responsive styling
Frontend	Lucide React	Icon set for findings badges, UI chrome
Frontend	Axios / Fetch API	REST API client with JWT auth headers
Backend	Python 3.11 + FastAPI	REST API, async request handling
Backend	SQLAlchemy 2.0	ORM — models, relationships, migrations
Backend	Pydantic v2 + pydantic-settings	Request validation, config from env vars
Backend	python-jose + bcrypt	JWT auth tokens, password hashing
Database	PostgreSQL (Render)	Primary data store — users, drafts, findings, audit log
AI / NLP	Google Gemini 2.5 Flash	Tone analysis, claim explanation, suggestions
AI / NLP	TF-IDF cosine similarity	Semantic search over guideline chunks (offline fallback)
Rule Engine	Custom Python module	Deterministic checks: banned terms, claims, facts, palette
Deployment	Render (Web Service + Static)	Backend + frontend hosting, auto-deploy
Deployment	GitHub Actions (auto)	CI: Render triggers deploy on push to main

3.2 System Architecture

The platform follows a clean three-tier architecture:

Browser (React SPA) → REST API (FastAPI) → PostgreSQL

The review pipeline works in two stages:
1. Rule Engine (deterministic) — runs first, always, offline. Checks banned terms, unsupported claims, palette.
2. AI Layer (Gemini 2.5 Flash) — runs after rule engine if API key is configured. Analyses tone, semantics, and generates natural-language suggestions citing retrieved guideline clauses.

If no AI key is set, the system falls back to TF-IDF semantic search + rule engine only — still fully functional.

3.3 Key Implementation Decisions

- **Lifespan-based DB init:** FastAPI lifespan context manager runs SQLAlchemy Base.metadata.create_all() on every cold start. Idempotent — safe to run repeatedly. Eliminates the need for external migration scripts on Render.
- **Offline-first rule engine:** All deterministic checks (banned terms, claim patterns, fact matching) run without any external API. The platform is fully functional even without a Gemini API key.
- **Per-finding provenance:** is_ai_generated bool + confidence float stored on every Finding row so the UI can accurately badge each finding as AI or Rule Engine regardless of which path generated it.
- **Workspace isolation:** All data (brand kit, drafts, findings, audit log) is scoped to workspace_id. Users can only access data within their workspace — enforced at the ORM query level, not just the route level.
- **Immutable audit log:** AuditLog rows are insert-only. No update or delete endpoint exists. Stores user_id, action, entity_type, entity_id, ai_service_used for full traceability.
- **LF line-ending enforcement:** .gitattributes forces backend/start.sh to LF on Windows to prevent bash parse failures on Render's Linux containers.

4. Changes from the Original Plan

The Round 1 plan was largely delivered as designed. The changes below reflect practical decisions made during the build phase:

Original Plan	What Changed	Reason
pgvector for semantic search	TF-IDF cosine similarity (Python, in-process)	Render free tier does not support pgvector extension. TF-IDF delivers comparable results for guideline retrieval at this scale with zero infrastructure overhead.
Alembic for DB migrations	SQLAlchemy create_all() in FastAPI lifespan	Alembic requires a migration history file checked into the repo and manual revision management. For a hackathon deploy, create_all() is simpler, idempotent, and reliable.
Alt-text / contrast ratio checks (image accessibility)	Colour-palette compliance check implemented; alt-text check deferred to Round 3	Colour-palette check delivers immediate visible value. Full accessibility (alt text, contrast ratio) requires a vision API call per image — planned for Round 3 with Gemini Vision.
Canva / Google Docs integration	Not implemented in Round 2	Integration with external publishing tools is a Round 3 / post-event feature. Core review workflow was prioritised.
Suggestion shown as a visual diff	Suggestion text shown inline per finding; full side-by-side diff UI deferred to Round 3	Inline suggestions are immediately usable. A polished diff view requires a dedicated diff library and additional UI work — scheduled for Round 3.
Primary research (survey + interviews)	Survey results referenced in context; direct user testing to be completed before final	Build phase was prioritised to have a working product. User testing session scheduled during

	submission	Round 3 preparation.

5. Three Major Improvements for Round 3

Improvement 1 — Gemini Vision Image Review & Accessibility Checks

What we will build: Full image analysis using Gemini Vision API — logo placement and colour correctness, brand colour palette compliance with hex-level detail, alt-text presence check, contrast ratio calculation for accessibility (WCAG 2.1 AA), and flagging of off-brand visual elements.

Why it matters:

- Visual content is a primary vector for brand inconsistency — wrong logo versions, off-palette colours, missing accessibility metadata.
- Extends BrandGuard AI beyond text to cover the full range of publishable assets.
- Directly addresses the gap identified in our Round 1 plan under "Asset Rules".

Deliverable: Image upload → AI visual analysis → findings panel with annotated image preview and severity badges.

Improvement 2 — Revision History with Visual Diff & Inline Suggestions

What we will build: A side-by-side diff view between any two draft versions, showing additions, deletions and unchanged text highlighted. Each finding's suggestion will appear as an inline accept/reject control that applies the change directly to the draft content. Version timeline visible in a sidebar.

Why it matters:

- Reviewers currently cannot see what changed between version 1 and version 2 without manually comparing text.
- One-click suggestion acceptance dramatically reduces the time for creators to act on findings.
- Gives managers a clear audit trail: who changed what, when, and in response to which finding.

Deliverable: Version comparison UI, inline accept/apply suggestion controls, version timeline sidebar.

Improvement 3 — Guideline-Gap Insights & Recurring Issues Dashboard

What we will build: An analytics layer that aggregates finding patterns across all drafts and surfaces: (a) which guidelines are triggered most often — suggesting the guideline is unclear or unknown to the team; (b) which team members or content types generate the most violations; (c) a "guideline health score" that shows how well the brand kit is working. The insights panel will suggest which guideline sections need rewording or additional examples.

Why it matters:

- The current dashboard shows counts but not actionable patterns.
- If the same violation appears 20 times across creators, the problem is likely the guideline — not the creator.
- Closes the loop from enforcement → learning → guideline improvement.

Deliverable: Insights page with heatmap of violations by guideline section, creator, and content type; guideline-gap recommendations.

6. Responsible Design — Privacy, Security & Human Oversight

- **Human control:** AI only suggests. Every finding requires a human to accept, dismiss or act. Nothing is auto-applied.
- **Explainability:** No finding appears without a reason, a guideline citation, and a confidence indicator.
- **AI transparency:** Every finding is badged with its source (Rule Engine or Gemini AI). Users always know what generated a flag.
- **Privacy:** Brand kits are isolated per workspace. Content is not sent to any model for training. Gemini API calls are stateless inference only.
- **Security:** JWT auth with bcrypt-hashed passwords, HTTPS-only deployment, role-based access enforced at ORM level, immutable audit log.
- **Reliability:** Low-confidence AI findings are marked accordingly. The rule engine runs first and independently for deterministic guarantees.
- **Accessibility:** UI follows accessible design — WCAG-compliant colour contrast, keyboard navigation, labelled form controls.

7. Team & Contributions

Member	Role	Round 2 Contribution
Ajay Kale (Team Leader)	Backend & AI Lead	FastAPI backend, SQLAlchemy models, rule engine, Gemini AI integration, JWT auth, database schema, Render deployment, CI/CD setup, API debugging
Mahesh Gadekar	Frontend & UX Lead	React SPA, Tailwind UI, findings panel, dashboard charts, role-based routing, image upload UI, responsive layout, dark mode
Asma Inamdar	Research & QA Lead	User testing coordination, QA across all features, bug reporting, findings filter UX, export feature testing, demo data review
Shrushti Kharadkar	Documentation & Demo Lead	Submission document, demo walkthrough script, feature list, Round 3 planning, presentation slide deck preparation

Field	Details
Team Name	Innovex
Team Leader	Ajay Kale
Leader Email	kaleajay626@gmail.com
Members	Ajay Kale, Mahesh Gadekar, Asma Inamdar, Shrushti Kharadkar
Live Project	https://brandguard-frontend-6swq.onrender.com
Source Code	https://github.com/AJJU3836/BrandGuard-AI

Google Drive	https://drive.google.com/drive/folders/1ybSTBQjKro8z8dBaycZW9Flvnrpbdp3r